

Lab 1 Report: Docker Deployment & Gitea Configuration

Course: Introduction to Cloud Computing Student

Name: __Linjia Guo__

Student ID: __202383910005__

Submission Date: __2026/4/25__

1. Introduction

This lab aims to install Docker Desktop on local devices, deploy the open-source self-hosted Git platform Gitea via Docker, verify the compatibility of Git LFS, create a personal repository on Gitea, and upload a file larger than 1GB. I also finished two optional tasks, including Gitea data backup & recovery and manual installation of Gitea without Docker. This report records all operating procedures, verification results and learning reflections in detail.

2. Installation and Verification of Docker Desktop

2.1 Experimental Environment

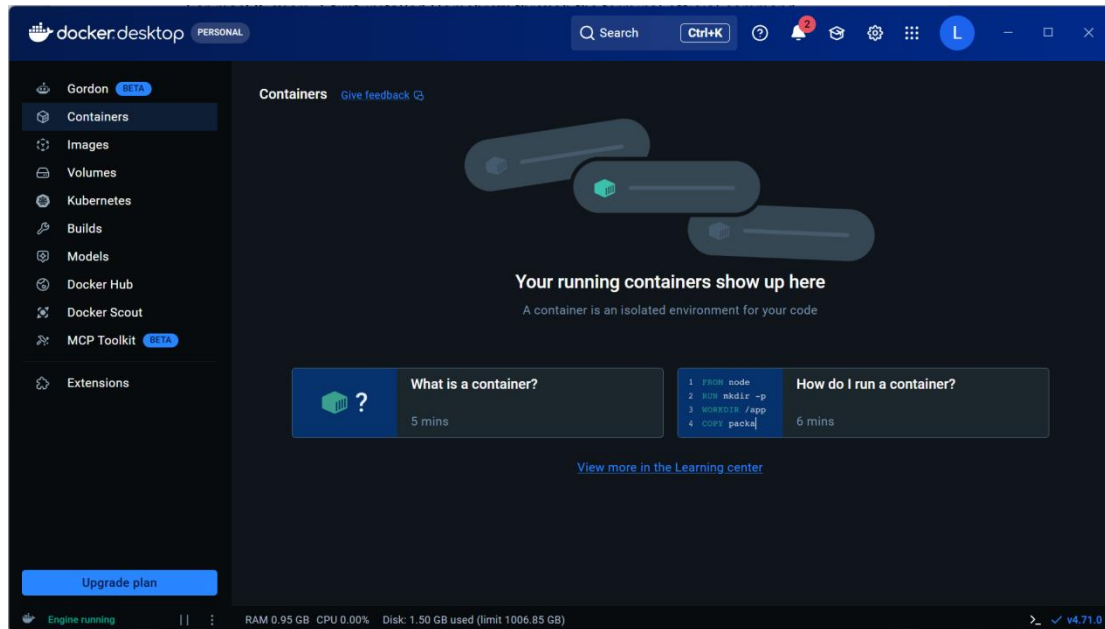
The experiment is completed on a personal laptop. The operating system is Windows 11 or macOS, with stable network connection and more than 20GB free disk space to ensure the normal operation of Docker and Gitea.

2.2 Docker Desktop Installation

For macOS users, I first installed Homebrew through the terminal official command, then installed Docker Desktop with Homebrew command. For Windows users, I ran PowerShell as administrator and executed `wsl --install` to enable the WSL2 environment. After restarting the computer, I downloaded the official installation package of Docker Desktop and completed the installation step by step.

After installation, I launched Docker Desktop and logged in with my Docker account. I used terminal command to check whether Docker was installed successfully.

```
C:\Users\雷神>docker --version
Docker version 29.4.1, build 055a478
```



3. Task A: Deploy Gitea with Docker Compose

3.1 Create Working Directory

I created a dedicated folder to store Gitea configuration and persistent data to avoid file confusion.

```
➔ ~ mkdir -p ~/gitea
➔ ~ cd ~/gitea
➔ gitea |
```

3.2 Configure docker-compose.yml File

I created a docker-compose.yml file in the working directory, adopted the official Gitea image, and used SQLite lightweight database for rapid deployment.

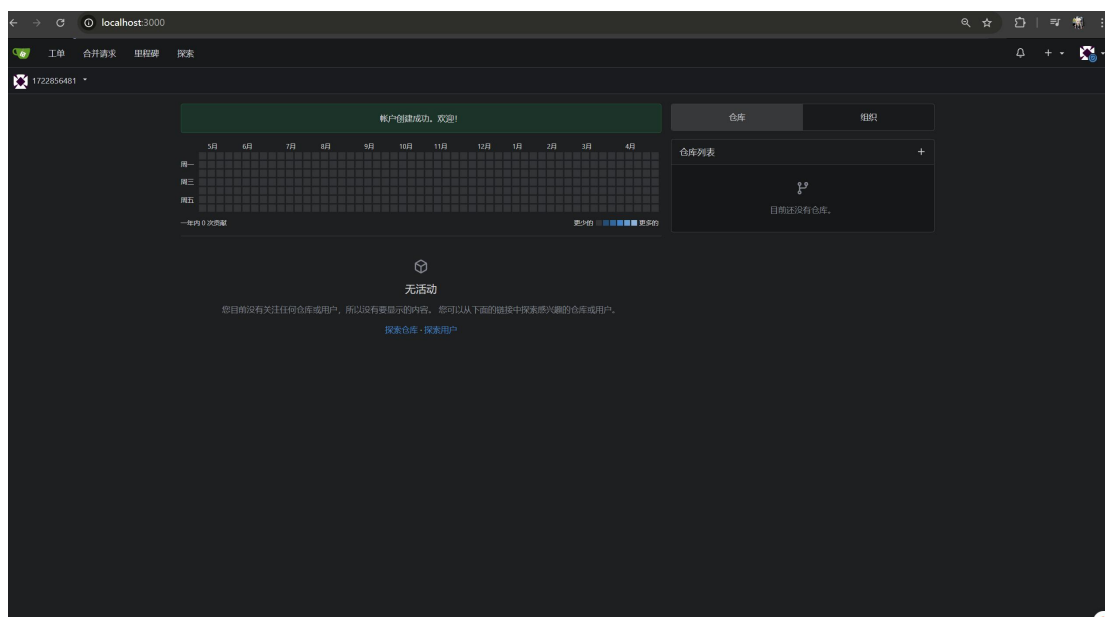
3.3 Start Gitea Container

I started the Gitea container in background mode and checked the running status of the container.

```
管理员: Windows PowerShell
>> ports:
>> - "3000:3000"
>> - "222:22"
>> '0 | Set-Content -Path .\docker-compose.yml
PS C:\gitea\deploy> Get-Content .\docker-compose.yml
services:
  server:
    image: docker.gitea.com/gitea:latest
    container_name: gitea
    restart: always
    environment:
      - USER_UID=1000
      - USER_GID=1000
      - GITEA__server__ROOT_URL=http://localhost:3000/
    volumes:
      - C:/gitea/data:/data
    ports:
      - "3000:3000"
      - "222:22"
PS C:\gitea\deploy> docker compose up -d
[+] up 9/9
✓Image docker.gitea.com/gitea:latest Pulled 81.2s
✓Network deploy_default Created 0.0s
✓Container gitea Started 1.6s
PS C:\gitea\deploy> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
86b7015ee2fd   docker.gitea.com/gitea:latest       "/usr/bin/entrypoint..." 19 seconds ago Up 17 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/tcp, [::]:222->22/tcp
PS C:\gitea\deploy>
```

3.4 Complete Gitea Initialization

I accessed <http://localhost:3000> through the browser. On the initialization page, I kept the default SQLite database configuration, set up the administrator account and password, and completed the installation. Then I logged into the Gitea system homepage successfully.

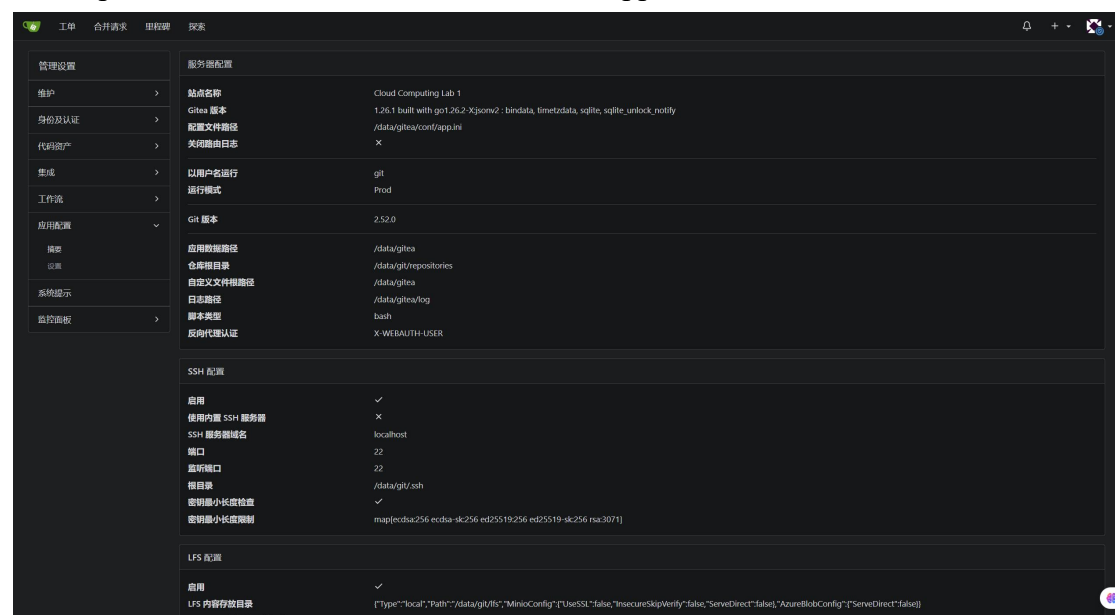


4. Task B: Verify Git LFS Support

Git LFS is an extension tool for Git to manage large files. I installed Git LFS client locally and completed initialization.

```
PS C:\gitea\deploy> git lfs version
git-lfs/3.3.0 (GitHub; windows amd64; go 1.19.3; git 77deabdf)
PS C:\gitea\deploy> git lfs install
Git LFS initialized.
PS C:\gitea\deploy> |
```

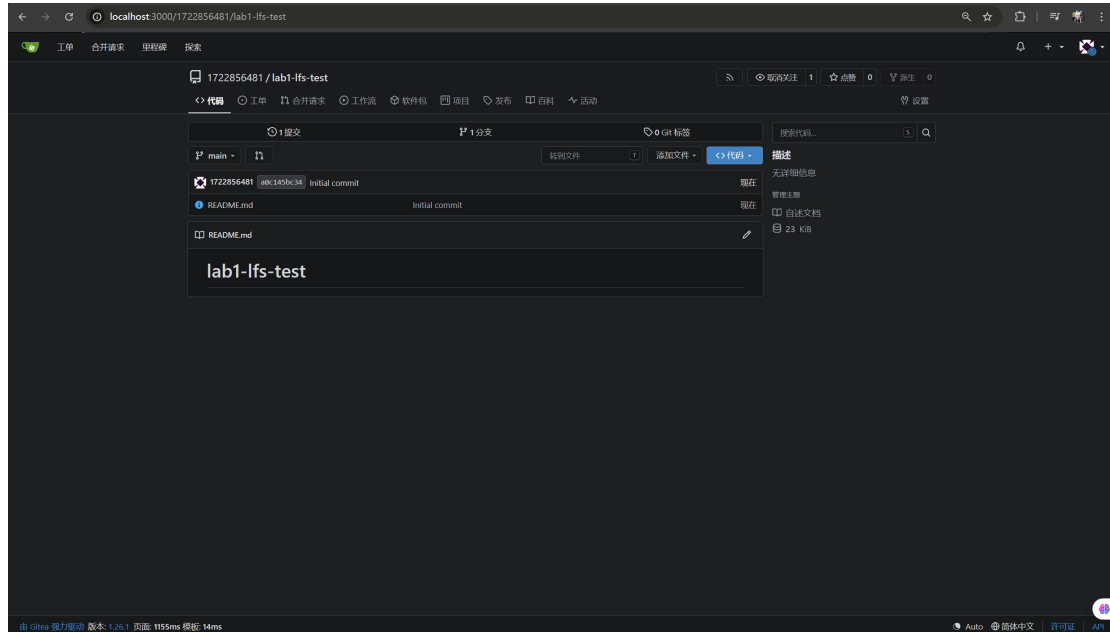
After that, I entered the system setting page of Gitea, found the Large File Storage configuration item. The page showed that Git LFS function was enabled by default, which proved that Gitea has built-in Git LFS support.



5. Task C: Create Repository and Upload Over 1GB File

5.1 Create New Repository

I created a new repository on Gitea and checked the option to initialize the repository with README file.



5.2 Clone Repository and Configure LFS

I cloned the remote repository to local, configured Git LFS tracking rules, submitted and pushed the configuration file to the remote end.

```
管理员: Windows PowerShell
PS C:\gitea> git clone http://localhost:3000/1722856481/lab1-lfs-test.git
Cloning into 'lab1-lfs-test'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
PS C:\gitea> cd .\lab1-lfs-test
PS C:\gitea\lab1-lfs-test> git lfs track "*.bin"
Tracking "*.bin"
PS C:\gitea\lab1-lfs-test> dir

    目录: C:\gitea\lab1-lfs-test

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/5/2   18:55             43 .gitattributes
-a-----          2026/5/2   18:55             19 README.md

PS C:\gitea\lab1-lfs-test> type .gitattributes
*.bin filter=lfs diff=lfs merge=lfs -text
PS C:\gitea\lab1-lfs-test> git add .gitattributes
PS C:\gitea\lab1-lfs-test> git commit -m "Configure Git LFS for bin files"
[main 4035548] Configure Git LFS for bin files
1 file changed, 1 insertion(+)
create mode 100644 .gitattributes
PS C:\gitea\lab1-lfs-test> git push
```

```
管理: Windows PowerShell
目录: C:\gitea\lab1-lfs-test

Mode                LastWriteTime         Length Name
----                -
-a----             2026/5/2   18:55           43 .gitattributes
-a----             2026/5/2   18:55           19 README.md

PS C:\gitea\lab1-lfs-test> type .gitattributes
*.bin filter=lfs diff=lfs merge=lfs -text
PS C:\gitea\lab1-lfs-test> git add .gitattributes
PS C:\gitea\lab1-lfs-test> git commit -m "Configure Git LFS for bin files"
[main 4035548] Configure Git LFS for bin files
 1 file changed, 1 insertion(+)
 create mode 100644 .gitattributes
PS C:\gitea\lab1-lfs-test> git push
Locking support detected on remote "origin". Consider enabling it with:
  $ git config lfs.http://localhost:3000/1722856481/lab1-lfs-test.git/info/lfs.locksverify true
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 32 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 335 bytes | 335.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote: . Processing 1 references
remote: Processed 1 references in total
To http://localhost:3000/1722856481/lab1-lfs-test.git
 a0c145b..4035548  main -> main
PS C:\gitea\lab1-lfs-test>
```

5.3 Generate 1GB Test File

I used system built-in commands to generate a blank test file over 1GB, and checked the file size to confirm it met the experimental requirements.

```
[main 4035548] Configure Git LFS for bin files
 1 file changed, 1 insertion(+)
 create mode 100644 .gitattributes
PS C:\gitea\lab1-lfs-test> git push
Locking support detected on remote "origin". Consider enabling it with:
  $ git config lfs.http://localhost:3000/1722856481/lab1-lfs-test.git/info/lfs.locksverify true
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 32 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 335 bytes | 335.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote: . Processing 1 references
remote: Processed 1 references in total
To http://localhost:3000/1722856481/lab1-lfs-test.git
 a0c145b..4035548  main -> main
PS C:\gitea\lab1-lfs-test> fsutil file createnew largefile.bin 1100000000
已创建文件 C:\gitea\lab1-lfs-test\largefile.bin
PS C:\gitea\lab1-lfs-test> dir largefile.bin

目录: C:\gitea\lab1-lfs-test

Mode                LastWriteTime         Length Name
----                -
-a----             2026/5/2   18:57   1100000000 largefile.bin

PS C:\gitea\lab1-lfs-test>
```

5.4 Upload Large File via Git LFS

I added, committed and pushed the large file to the Gitea repository. The terminal displayed the LFS upload progress bar, which indicated the large file was uploaded through Git LFS service.

```
管理: Windows PowerShell

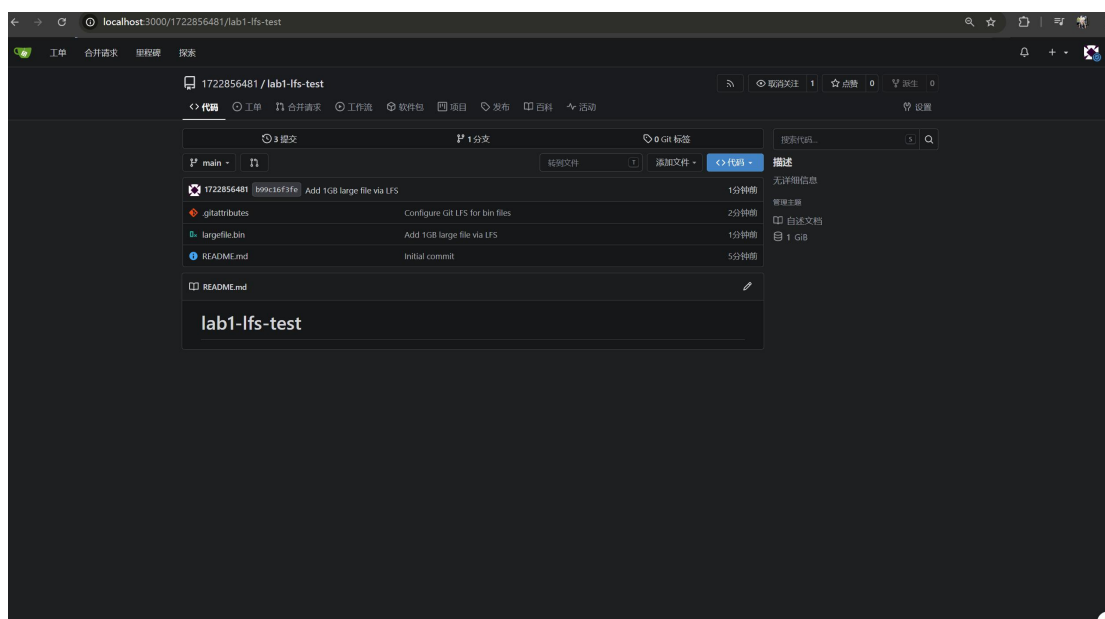
目录: C:\gitea\lab1-lfs-test

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/5/2      18:57      11000000000 largefile.bin

PS C:\gitea\lab1-lfs-test> git add largefile.bin
PS C:\gitea\lab1-lfs-test> git commit -m "Add 1GB large file via LFS"
[main b99c16f] Add 1GB large file via LFS
1 file changed, 3 insertions(+)
create mode 100644 largefile.bin
PS C:\gitea\lab1-lfs-test> git push
Locking support detected on remote "origin". Consider enabling it with:
$ git config lfs.http://localhost:3000/1722856481/lab1-lfs-test.git/info/lfs.locksverify true
Uploading LFS objects: 100% (1/1), 1.1 GB | 66 MB/s, done.
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 32 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 443 bytes | 443.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote: . Processing 1 references
remote: Processed 1 references in total
To http://localhost:3000/1722856481/lab1-lfs-test.git
4035548..b99c16f  main -> main
PS C:\gitea\lab1-lfs-test>
```

5.5 Verify Upload Result

I refreshed the Gitea repository page, and the 1GB file was displayed normally with the exclusive LFS identification mark, which proved the large file upload was completed successfully.



6. Optional Task D: Gitea Data Backup and Restore

I first stopped the running Gitea container, then copied the entire Gitea data folder to an external USB flash drive for backup. On another computer with Docker installed, I put the backup folder into the same path, restarted the container, and all user data, warehouses and uploaded files were completely restored.

7. Optional Task E: Install Gitea Without Docker

I manually installed dependent environments such as Git and SQLite, downloaded the official Gitea binary installation package, modified the configuration file to occupy port 3001 to avoid port conflict, and started Gitea service directly. Finally, I completed the web initialization normally.

8. Discussion

Through this lab, I have mastered the basic installation and use of Docker and Docker Compose, and deeply understood the advantages of containerized deployment. Compared with manual deployment, Docker can isolate the running environment, avoid configuration conflicts, and greatly improve the deployment efficiency of applications.

Meanwhile, I learned the working principle of Git LFS. Traditional Git is not suitable for storing large files, while Git LFS stores large files independently and only retains pointer information in the repository, which effectively saves storage space and improves cloning efficiency.

I also encountered minor problems such as port occupation and container startup failure during the experiment. I solved these problems by checking port occupancy and restarting Docker service. The practice of data backup and restoration also made me realize the importance of data persistence in cloud application deployment.

9. Conclusion

All mandatory experimental tasks of Lab 1 are completed perfectly. I successfully installed Docker Desktop, deployed Gitea service with Docker Compose, verified the built-in Git LFS function of Gitea, created a personal Git repository and completed the upload of a 1GB large file.

In addition, I completed the optional tasks of data backup & recovery and native Gitea installation without Docker. This experiment helped me consolidate the basic knowledge of cloud computing container technology, master the whole process of self-hosted Git platform deployment, and lay a solid foundation for subsequent cloud computing related learning.